Cahier des charges – App d'épargne par arrondis

Nov 7, 2026 · @Timothée Benchergui

1. Objet et périmètre

Ce document définit la version 1 d'une application privée qui calcule, pour chaque paiement par carte du compte bancaire d'un utilisateur, une somme d'épargne obtenue par arrondi, et l'affiche dans un tableau de bord. Il s'appuie sur l'étude de marché et les choix technologiques, dont il reprend la stack sans la remettre en cause.

Dans le périmètre de la version 1

- Connexion en lecture seule d'un compte bancaire par utilisateur, via Enable Banking en mode production restreint, pour les banques de la section 9.6 (BoursoBank, Crédit Agricole, Crédit Mutuel, La Banque Postale).
- Synchronisation quotidienne des transactions et calcul automatique des arrondis.
- Tableau de bord : total épargné, épargne du mois, montant à virer, historique mensuel, liste des transactions.
- Enregistrement manuel du virement mensuel vers l'épargne.
- Un objectif d'épargne par utilisateur.
- Réglages d'arrondi par utilisateur (palier, multiplicateur, plafonds).
- Application web progressive installable sur iPhone et Android.
- Gestion des utilisateurs par un administrateur.
- Export et suppression des données de chaque utilisateur.

Hors périmètre : voir la section 14.

Règle de lecture. Dans ce document, « doit » marque une exigence obligatoire, « devrait » une exigence recommandée, « peut » une option. Toute valeur chiffrée est la valeur retenue, sauf mention « à confirmer », qui renvoie à un jalon précis de la section 13.

2. Acteurs et utilisateurs

L'application compte au plus 10 utilisateurs en version 1, chacun avec ses propres données, strictement isolées des autres.

Acteur	Qui	Nombre	Droits
Administrateur	L'auteur	1	Crée, désactive et réactive des utilisateurs. Possède aussi un espace de membre ordinaire. Ne peut lire aucune donnée financière d'un autre utilisateur via l'application.
Membre	Les utilisateurs créés par l'administrateur	4	Lit et modifie uniquement ses propres données.
Système	Tâches planifiées de l'API	2	Synchronisation quotidienne des transactions, purge mensuelle des données anciennes.

Services externes. Enable Banking (accès aux données bancaires), BoursoBank (la banque), Cloudflare (tunnel et authentification).

Appareils ciblés. iPhone (cible principale) et Android. Versions minimales : iOS 16.4 et Android avec Chrome à jour (valeur à confirmer sur les appareils réels, voir D-11).

Règle d'isolement. Aucune route de l'API ne renvoie une donnée rattachée à un autre utilisateur que celui identifié. Un accès par identifiant à une ressource d'un autre utilisateur renvoie 404, jamais 403, pour ne pas révéler son existence.

3. Glossaire et conventions

Chaque terme ci-dessous a un seul sens dans tout le document.

Terme	Définition exacte
Transaction	Une opération du compte bancaire de l'utilisateur renvoyée par Enable Banking.
Transaction éligible	Une transaction qui remplit toutes les conditions de la section 4.2 et donne donc lieu à un arrondi.

Terme	Définition exacte
Palier	Le multiple, en euros, auquel on arrondit le montant d'une transaction (1, 2, 5, 10 € ou une valeur personnalisée).
Multiplicateur	Un entier par lequel l'arrondi de base est multiplié (1, 2, 3, 5 ou 10).
Arrondi de base	Différence entre le montant arrondi au palier supérieur et le montant réel.
Arrondi final	Arrondi de base multiplié, puis borné par les plafonds. C'est la somme mise de côté pour la transaction.
Total épargné	Somme des arrondis finaux de toutes les transactions éligibles depuis la date d'activation du compte.
Épargne du mois	Somme des arrondis finaux des transactions dont la date de comptabilisation tombe dans le mois civil courant.
Virement enregistré	Un virement vers l'épargne que l'utilisateur déclare avoir fait lui-même hors de l'application.
Montant à virer	Total épargné moins la somme des virements enregistrés. Jamais négatif : s'il est négatif, il vaut 0.
Date d'activation	Date et heure de la fin du premier consentement bancaire réussi. Seules les transactions comptabilisées à partir de cette date comptent.
Consentement	L'autorisation donnée à la banque d'ouvrir ses données à l'application, valable pour une durée limitée.
Application Enable Banking	L'application que chaque membre enregistre dans le panneau de contrôle d'Enable Banking ; elle possède un identifiant et une clé privée.
Mois	Mois civil dans le fuseau Europe/Paris.

Conventions techniques

- **Montants** : entiers en centimes d'euro, dans la base, l'API et le code. Aucun nombre à virgule pour de l'argent. Les champs d'API concernés se terminent par `_cents`.
- **Devise** : euro uniquement.

- **Affichage** : format français, par exemple `1 234,56 €`, avec espace insécable.
- **Dates** : `AAAA-MM-JJ` dans l'API pour une date de calendrier ; horodatages en UTC au format RFC 3339 stockés en `timestamptz`.
- **Fuseau de référence** : Europe/Paris pour découper les mois et déclencher les tâches planifiées.
- **Identifiants** : UUID.
- **Langue** : français uniquement.

4. Règles de calcul des arrondis

L'arrondi final d'une transaction est calculé une seule fois, à l'enregistrement de la transaction, puis il n'est plus jamais modifié. Tous les calculs se font en centimes entiers.

4.1 Paramètres par utilisateur

Paramètre	Valeurs permises	Défaut
Palier `p`	1 €, 2 €, 5 €, 10 €, ou valeur personnalisée de 0,50 € à 50,00 € par pas de 0,50 €	1,00 €
Multiplicateur `k`	1, 2, 3, 5 ou 10	1
Plafond par transaction `Ptx`	de 1,00 € à 100,00 € par pas de 1,00 €	20,00 €
Plafond mensuel `Pmois`	aucun, ou de 5,00 € à 1 000,00 € par pas de 5,00 €	aucun

Toute valeur hors de ces plages est refusée par l'API avec le code 422.

4.2 Conditions d'éligibilité

Une transaction est éligible si et seulement si les sept conditions suivantes sont vraies :

1. Le sens est un débit.
2. Le statut est comptabilisé (la transaction n'est plus en attente).
3. La devise est l'euro.
4. Le montant est strictement positif.
5. La date de comptabilisation est égale ou postérieure au jour de la date d'activation (fuseau Europe/Paris).
6. C'est un paiement par carte, et non un retrait d'espèces (méthode de détection : D-06).
7. Le compte est suivi.

Une transaction non éligible est quand même enregistrée, avec une raison d'exclusion et sans arrondi :

Code	Cas
`credit`	Le sens est un crédit (dont les remboursements, voir D-07)
`foreign_currency`	La devise n'est pas l'euro
`zero_amount`	Le montant est nul
`before_activation`	La date de comptabilisation précède la date d'activation
`not_card`	Ce n'est pas un paiement par carte
`cash_withdrawal`	C'est un retrait d'espèces
`untracked_account`	Le compte n'est pas suivi

4.3 Formule

Soit `m` le montant en centimes, `p` le palier en centimes, `k` le multiplicateur.

```
\text{base} = (p - (m \bmod p)) \bmod p
\text{brut} = \text{base} \times k
\text{final} = \min(\text{brut},\ P_{tx},\ R_{mois})
```

`R_mois` est le reste du plafond mensuel : `Pmois` moins la somme des arrondis finaux déjà enregistrés pour le mois de la transaction, borné à 0 au minimum. Sans plafond mensuel, `R_mois` est illimité. Quand un montant est déjà un multiple exact du palier, l'arrondi de base vaut 0 (D-02). Les transactions d'un même mois sont traitées dans l'ordre de leur enregistrement dans l'application.

4.4 Exemples de référence (devenant des tests, section 13)

Montant	Palier	Mult.	Base	Brut	Final	Remarque
2,50 €	1 €	1	0,50 €	0,50 €	0,50 €	Cas simple
2,00 €	1 €	1	0	0	0	Multiple exact du palier
10,00 €	10 €	5	0	0	0	Multiple exact, multiplicateur sans effet
12,35 €	5 €	1	2,65 €	2,65 €	2,65 €	Palier de 5 €
12,35 €	10 €	2	7,65 €	15,30 €	15,30 €	Multiplicateur 2
47,10 €	10 €	3	2,90 €	8,70 €	8,70 €	Multiplicateur 3
8,10 €	10 €	10	1,90 €	19,00 €	19,00 €	Juste sous le plafond par transaction

Montant	Palier	Mult.	Base	Brut	Final	Remarque
3,10 €	10 €	10	6,90 €	69,00 €	20,00 €	Plafond par transaction de 20,00 € atteint
3,20 €	0,50 €	1	0,30 €	0,30 €	0,30 €	Palier personnalisé
19,99 €	1 €	1	0,01 €	0,01 €	0,01 €	Plus petit arrondi possible
0,01 €	1 €	1	0,99 €	0,99 €	0,99 €	Plus grand arrondi de base pour un palier de 1 €
3,10 €	1 €	10	0,90 €	9,00 €	2,00 €	Plafond mensuel de 50,00 €, déjà 48,00 € cumulés ce mois-ci

4.5 Autres règles

- **Changement de réglage** : il ne s'applique qu'aux transactions enregistrées après le changement ; aucun recalcul rétroactif (D-08).
- **Début du comptage** : aucune transaction antérieure à la date d'activation n'est comptabilisée (D-09).
- **Remboursements** : un crédit n'a aucun effet sur les arrondis déjà enregistrés (D-07).
- **Mois d'une transaction** : celui de sa date de comptabilisation, lue comme une date de calendrier sans heure.
- **Transaction en attente devenue comptabilisée** : elle n'est ni lue ni enregistrée tant qu'elle est en attente ; elle est enregistrée et arrondie une seule fois, quand la banque la renvoie comme comptabilisée.

5. Exigences fonctionnelles

Chaque exigence est vérifiable par un test. Priorité : « Doit » est obligatoire pour la version 1, « Devrait » est recommandée.

Identité et accès

Id	Exigence	Critère d'acceptation	Priorité
EF-01	L'API identifie l'utilisateur uniquement par le JWT Cloudflare Access, dont elle vérifie la signature, l'audience et l'expiration	Une requête sans JWT, avec un JWT expiré ou mal signé reçoit 401	Doit
EF-02	L'API rattache le JWT à un utilisateur préenregistré par son email	Un email inconnu ou un compte désactivé reçoit 403	Doit
EF-03	Un utilisateur n'accède qu'à ses propres données	Demander la ressource d'un autre utilisateur renvoie 404	Doit

Id	Exigence	Critère d'acceptation	Priorité
EF-04	L'administrateur crée un utilisateur (email, nom), le désactive et le réactive	Un utilisateur désactivé reçoit 403 à sa prochaine requête ; ses données sont conservées	Doit

Connexion bancaire et synchronisation

Id	Exigence	Critère d'acceptation	Priorité
EF-05	L'utilisateur enregistre l'identifiant et la clé privée de son application Enable Banking	La clé est chiffrée avant stockage (section 10) et n'est jamais renvoyée par l'API	Doit
EF-06	L'utilisateur choisit sa banque, démarre l'autorisation chez elle et l'API traite le retour	L'état aléatoire est lié à l'utilisateur, à usage unique et valable 10 minutes ; un état inconnu ou expiré est refusé	Doit
EF-07	L'application affiche l'état de la connexion	Cinq états : aucune, active (avec date de fin), expire bientôt (14 jours ou moins), expirée, en erreur	Doit
EF-08	L'utilisateur révoque sa connexion	La session Enable Banking est supprimée si possible ; la clé et les jetons locaux sont effacés	Doit
EF-09	Une synchronisation automatique s'exécute chaque jour à 06:30 (Europe/Paris) pour chaque connexion active	Une ligne de suivi de synchronisation est créée par exécution	Doit
EF-10	L'utilisateur peut lancer une synchronisation manuelle	Maximum 1 par heure et 4 sur 24 heures glissantes ; au-delà, 429	Devrait
EF-11	Une synchronisation ne crée jamais de doublon	Relancer deux fois la même synchronisation laisse le nombre de transactions inchangé	Doit
EF-12	Une transaction en attente n'est jamais comptabilisée dans l'épargne	La synchronisation ne demande que les transactions comptabilisées ; une transaction en attente n'est ni enregistrée ni arrondie	Doit

Id	Exigence	Critère d'acceptation	Priorité
EF-13	Une synchronisation en échec ne modifie aucune donnée existante	Le suivi indique l'erreur ; l'écran affiche la date de la dernière synchronisation réussie	Doit
EF-37	L'utilisateur choisit sa banque parmi celles dont le profil est activable	La liste vient de GET /aspsps filtrée par les profils activables ; pour Crédit Agricole, une seconde liste propose les caisses régionales ; une banque sans profil validé est grisée avec la mention « Bientôt disponible »	Doit
EF-38	La détection d'un paiement par carte dépend de la banque de l'utilisateur	Le moteur d'arrondis interroge le profil de la banque ; un test unitaire par profil couvre au moins un paiement par carte, un retrait d'espèces et un virement	Doit

Arrondis

Id	Exigence	Critère d'acceptation	Priorité
EF-14	L'arrondi de chaque transaction éligible est calculé à l'enregistrement avec les réglages en vigueur	Les 12 exemples de la section 4.4 donnent exactement les valeurs attendues	Doit
EF-15	Un changement de réglage ne modifie aucun arrondi existant	Les arrondis enregistrés avant le changement restent identiques	Doit
EF-16	Chaque arrondi garde le palier, le multiplicateur, l'arrondi de base et l'arrondi final	Les quatre valeurs sont lisibles sur le détail d'une transaction	Doit
EF-17	Toute transaction non éligible est conservée avec sa raison d'exclusion	Les raisons suivent la liste de la section 4.2	Doit
EF-18	Le plafond mensuel, s'il est défini, borne l'arrondi final	Le total du mois ne dépasse jamais le plafond	Devrait

Tableau de bord, virements et objectif

Id	Exigence	Critère d'acceptation	Priorité
EF-19	L'accueil affiche le total épargné	Il est égal à la somme des arrondis finaux, au centime près	Doit
EF-20	L'accueil affiche l'épargne du mois	Elle suit la définition du glossaire (mois civil Europe/Paris, date de comptabilisation)	Doit
EF-21	L'accueil affiche le montant à virer	Total épargné moins virements enregistrés, jamais négatif	Doit
EF-22	L'historique mensuel liste les mois	12 mois par défaut, 24 mois au maximum, avec épargne et virement du mois	Doit
EF-23	La liste des transactions est paginée et filtrable par mois	50 lignes par page ; les transactions exclues sont visibles et marquées avec leur raison	Doit
EF-24	L'utilisateur enregistre un virement	Montant strictement positif, date non future, montant proposé par défaut égal au montant à virer	Doit
EF-25	L'utilisateur supprime un virement enregistré par erreur	Le montant à virer est recalculé immédiatement	Devrait
EF-26	L'utilisateur définit un objectif	Nom, montant cible strictement positif, date cible facultative ; un seul objectif	Doit
EF-27	L'application affiche la progression de l'objectif	Total épargné divisé par la cible, affiché en pourcentage entier, plafonné à 100 % à l'affichage	Doit
EF-28	L'utilisateur modifie ses réglages d'arrondi	Les valeurs hors plages de la section 4.1 sont refusées avec 422	Doit

Données personnelles et application

Id	Exigence	Critère d'acceptation	Priorité
EF-29	L'utilisateur exporte toutes ses données	Un fichier JSON contient réglages, transactions, arrondis, virements et objectif	Devrait

Id	Exigence	Critère d'acceptation	Priorité
EF-30	L'utilisateur supprime son compte et toutes ses données	Après confirmation, plus aucune ligne ne lui est rattachée en base	Doit
EF-31	Les transactions de plus de 24 mois sont purgées chaque mois	Les totaux mensuels agrégés sont conservés	Devrait
EF-32	L'application est installable sur l'écran d'accueil	Manifeste en affichage `standalone`, icônes 192 et 512 pixels ; installation vérifiée sur iPhone et Android	Doit
EF-33	L'application affiche une version limitée hors ligne	La structure de l'application et le dernier tableau de bord sont lisibles ; un bandeau indique « Hors ligne » ; aucune écriture n'est possible	Devrait
EF-34	L'application propose les mises à jour sans les imposer	Un bandeau propose de recharger ; aucun rechargement automatique	Devrait
EF-35	L'API expose un contrôle de santé	`GET /healthz` renvoie 200 si l'API et la base répondent, 503 sinon	Doit
EF-36	L'API journalise les actions sensibles	Connexion bancaire, révocation, suppression de compte et changement de réglage sont enregistrés dans un journal d'audit	Devrait

6. Modèle de données

Base PostgreSQL 18. Tous les identifiants sont des `uuid` générés par `gen_random_uuid()`. Toutes les dates d'horodatage sont des `timestamptz`. Toutes les clés étrangères vers `users` se suppriment en cascade (EF-30), sauf mention contraire. Les migrations sont gérées par `goose`.

`users`

Colonne	Type	Contraintes
`id`	uuid	Clé primaire
`email`	text	Non nul, unique, `CHECK (email = lower(email))`

Colonne	Type	Contraintes
`display_name`	text	Non nul, 1 à 60 caractères
`role`	text	Non nul, dans (`admin` , `member`)
`is_active`	boolean	Non nul, `true` par défaut
`created_at` , `updated_at`	timestamptz	Non nuls, `now()` par défaut

`user_settings` (une ligne par utilisateur, créée avec les valeurs par défaut)

Colonne	Type	Contraintes
`user_id`	uuid	Clé primaire, clé étrangère vers `users`
`step_cents`	integer	Non nul, 100 par défaut, entre 50 et 5000, multiple de 50
`multiplier`	integer	Non nul, 1 par défaut, dans (1, 2, 3, 5, 10)
`tx_cap_cents`	integer	Non nul, 2000 par défaut, entre 100 et 10000, multiple de 100
`month_cap_cents`	integer	Nul autorisé (aucun plafond), sinon entre 500 et 100000, multiple de 500
`updated_at`	timestamptz	Non nul

`bank_connections` (au plus une par utilisateur)

Colonne	Type	Contraintes
`id`	uuid	Clé primaire
`user_id`	uuid	Non nul, unique, clé étrangère
`eb_application_id`	text	Non nul
`eb_private_key_enc`	bytea	Non nul, clé privée chiffrée (section 10)
`eb_key_nonce`	bytea	Non nul, 12 octets
`eb_session_id`	text	Nul tant que l'autorisation n'est pas terminée
`aspsp_name`	text	Non nul ; nom exact de la banque (ou de la caisse régionale) dans GET /aspsps

Colonne	Type	Contraintes
`aspsp_country`	text	Non nul, FR en version 1
`consent_valid_until`	timestamptz	Nul tant que l'autorisation n'est pas terminée
`activated_at`	timestamptz	Date d'activation, fixée au premier consentement réussi et jamais modifiée ensuite
`status`	text	Non nul, dans (`pending` , `active` , `expired` , `error`)
`auth_state`	text	Nul hors autorisation en cours ; état aléatoire de 32 octets
`auth_state_expires_at`	timestamptz	Nul hors autorisation en cours
`last_sync_at` , `last_sync_ok_at`	timestamptz	Dernière tentative et dernier succès
`last_error`	text	Message sans donnée personnelle
`created_at` , `updated_at`	timestamptz	Non nuls

L'état « expire bientôt » n'est pas stocké : l'API le déduit quand `status` vaut `active` et que `consent_valid_until` est dans 14 jours ou moins.

`bank_accounts`

Colonne	Type	Contraintes
`id`	uuid	Clé primaire
`connection_id`	uuid	Non nul, clé étrangère
`eb_account_uid`	text	Non nul ; unique avec `connection_id`
`iban_last4`	char(4)	Quatre derniers caractères de l'IBAN ; l'IBAN complet n'est jamais stocké
`label`	text	Libellé du compte
`currency`	char(3)	Non nul
`is_tracked`	boolean	Non nul ; un seul compte suivi par utilisateur (index unique partiel)

`transactions`

Colonne	Type	Contraintes
`id`	uuid	Clé primaire
`account_id`	uuid	Non nul, clé étrangère ; en cascade avec la connexion
`external_id`	text	Non nul ; unique avec `account_id` (voir ci-dessous)
`booked_on`	date	Non nul, date de comptabilisation
`amount_cents`	bigint	Non nul, `>= 0`, valeur absolue du montant
`direction`	text	Non nul, dans (`debit` , `credit`)
`currency`	char(3)	Non nul
`label`	text	Non nul, libellé fourni par la banque
`bank_tx_code`	text	Code de transaction fourni par la banque, s'il existe
`is_eligible`	boolean	Non nul
`exclusion_reason`	text	Nul si éligible, sinon l'un des codes de la section 4.2
`raw`	jsonb	Non nul, réponse brute de la banque, supprimée avec la transaction
`created_at`	timestamptz	Non nul ; sert d'ordre d'enregistrement avec `id`

Index : (`account_id` , `booked_on` décroissant). `external_id` est l'identifiant de transaction fourni par la banque ; s'il manque, c'est l'empreinte SHA-256 de la date, du montant, du sens, du libellé et du rang dans la journée. La règle définitive est fixée par la fiche de mapping du jalon J1.

`roundings` (une ligne par transaction éligible, jamais modifiée)

Colonne	Type	Contraintes
`transaction_id`	uuid	Clé primaire, clé étrangère vers `transactions`
`user_id`	uuid	Non nul, clé étrangère ; sert aux requêtes et à l'isolement
`step_cents` , `multiplier`	integer	Réglages utilisés pour ce calcul
`base_cents`	integer	Non nul, `>= 0`

Colonne	Type	Contraintes
`gross_cents`	bigint	Non nul, `>= 0`
`final_cents`	bigint	Non nul, `>= 0`
`month`	date	Non nul, premier jour du mois de `booked_on` (Europe/Paris)
`created_at`	timestamptz	Non nul

Index : (`user_id` , `month`).

`transfers`

Colonne	Type	Contraintes
`id`	uuid	Clé primaire
`user_id`	uuid	Non nul, clé étrangère
`amount_cents`	bigint	Non nul, `> 0`
`transferred_on`	date	Non nul ; la non-futurité est contrôlée par l'API
`created_at`	timestamptz	Non nul

`goals` (au plus un par utilisateur)

Colonne	Type	Contraintes
`id`	uuid	Clé primaire
`user_id`	uuid	Non nul, unique, clé étrangère
`name`	text	Non nul, 1 à 60 caractères
`target_cents`	bigint	Non nul, `> 0`
`target_date`	date	Facultative
`created_at` , `updated_at`	timestamptz	Non nuls

`sync_runs`

Colonne	Type	Contraintes
`id`	uuid	Clé primaire

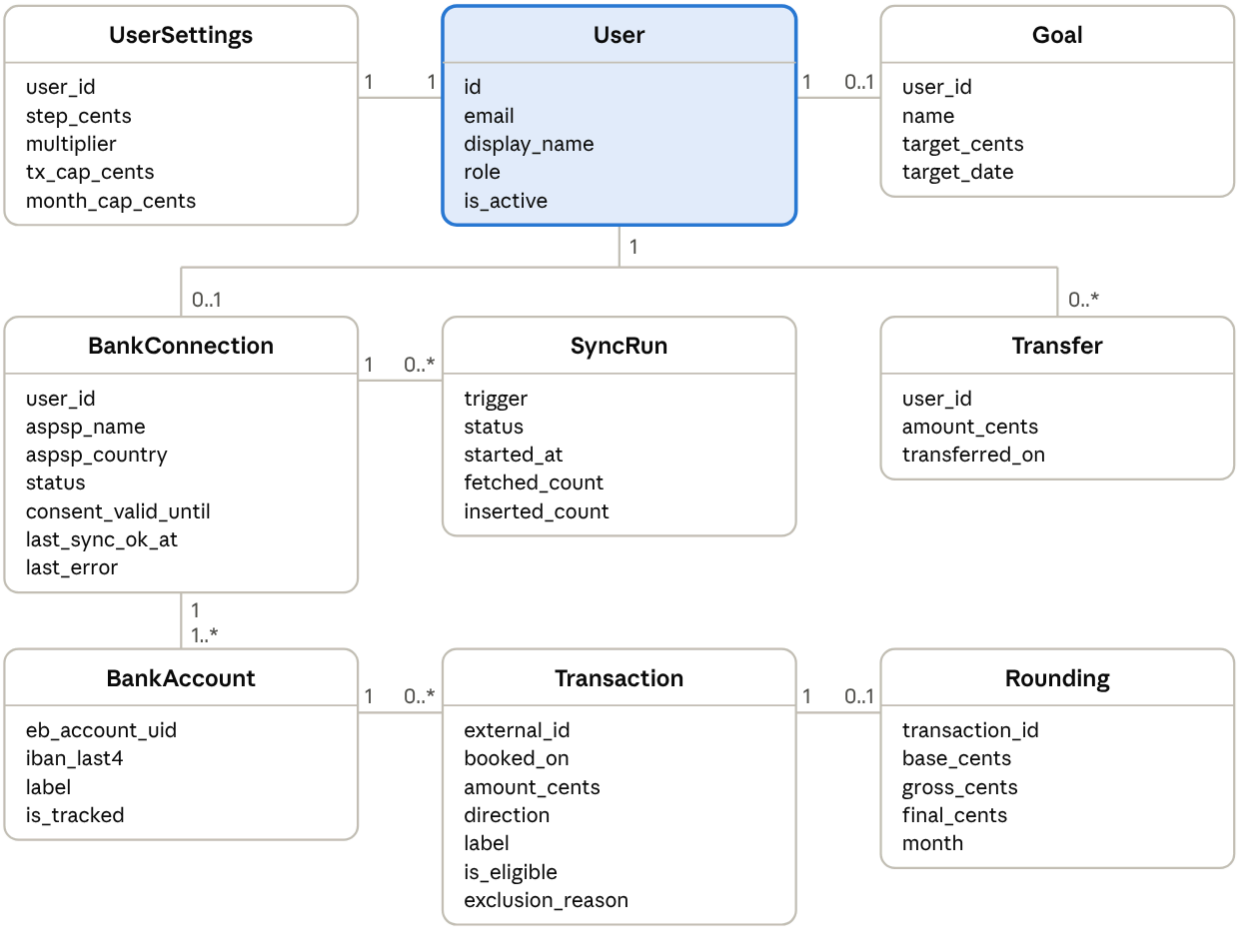
Colonne	Type	Contraintes
`connection_id`	uuid	Non nul, clé étrangère
`trigger`	text	Non nul, dans (`scheduled` , `manual`)
`status`	text	Non nul, dans (`running` , `success` , `error`)
`started_at` , `finished_at`	timestamptz	`finished_at` nul tant que l'exécution tourne
`fetched_count` , `inserted_count`	integer	Non nuls, 0 par défaut
`error`	text	Message sans donnée personnelle

`audit_log`

Colonne	Type	Contraintes
`id`	bigint	Clé primaire, identité générée
`user_id`	uuid	Nul autorisé ; clé étrangère `ON DELETE SET NULL`
`action`	text	Non nul
`detail`	jsonb	Ne contient ni email, ni libellé, ni montant
`created_at`	timestamptz	Non nul ; lignes conservées 12 mois

Contrainte de cohérence. La version 1 ne stocke que les transactions comptabilisées : la synchronisation ne demande à la banque que ce statut.

Tout part de l'utilisateur : réglages, banque, virements, objectif



Attributs principaux ; la liste complète est en section 6.

diagramme de classes du domaine · 9 entités

Diagramme de classes C1. Chaque utilisateur a une ligne de réglages, au plus une connexion bancaire et au plus un objectif. Une connexion porte un ou plusieurs comptes, dont un seul est suivi (`is_tracked`). Une transaction a au plus un arrondi.

7. Interface API REST

Tout est sous le préfixe `/api/v1` , en JSON UTF-8. Toutes les routes exigent le JWT Cloudflare Access (EF-01), sauf `GET /healthz` . Le contrat OpenAPI est généré depuis le code et fait foi ; le tableau ci-dessous en est le résumé normatif.

Conventions

- Les montants sont des entiers en centimes, dans des champs dont le nom se termine par `_cents` .
- Les dates de calendrier sont au format `AAAA-MM-JJ` ; les horodatages au format RFC 3339 en UTC.

- Les erreurs suivent le format `application/problem+json` (RFC 9457) avec les champs `type`, `title`, `status` et `detail`.
- Les listes sont paginées par curseur : paramètres `limit` (50 par défaut, 100 au maximum) et `cursor`.
- Codes d'erreur utilisés : 400 requête mal formée, 401 non authentifié, 403 compte inconnu ou inactif, 404 introuvable, 409 conflit, 422 valeur hors plage, 429 trop de requêtes, 502 la banque ou Enable Banking ne répond pas.

Méthode et route	Rôle	Entrée	Sortie et codes
`GET /healthz`	Aucune authentification, réseau interne uniquement	—	200 ou 503
`GET /me`	Membre	—	200 : `id`, `email`, `display_name`, `role`
`DELETE /me`	Membre	`confirm: true`	204 ; supprime le compte et toutes les données (EF-30)
`GET /me/export`	Membre	—	200 : fichier JSON (EF-29)
`GET /dashboard`	Membre	—	200 : voir l'exemple ci-dessous
`GET /months`	Membre	`limit` (12 par défaut, 24 au maximum)	200 : liste de `month`, `saved_cents`, `transferred_cents`, `transactions_count`
`GET /transactions`	Membre	`month` (`AAAA-MM`), `limit`, `cursor`	200 : liste avec, pour chaque ligne, la transaction et son arrondi ; 422 si `month` invalide
`GET /settings`	Membre	—	200 : réglages d'arrondi
`PUT /settings`	Membre	`step_cents`, `multiplier`, `tx_cap_cents`, `month_cap_cents`	200 ; 422 hors plages de la section 4.1

Méthode et route	Rôle	Entrée	Sortie et codes
`GET /goal`	Membre	—	200 ou 404 s'il n'y en a pas
`PUT /goal`	Membre	`name`, `target_cents`, `target_date`	200 ; 422 si `target_cents` n'est pas strictement positif
`DELETE /goal`	Membre	—	204
`GET /transfers`	Membre	`limit`, `cursor`	200 : virements, du plus récent au plus ancien
`POST /transfers`	Membre	`amount_cents`, `transferred_on`	201 ; 422 si le montant n'est pas strictement positif ou la date est dans le futur
`DELETE /transfers/{id}`	Membre	—	204 ; 404 si le virement n'est pas le sien
`GET /bank/connection`	Membre	—	200 : état de la connexion, dates, comptes ; 404 s'il n'y en a pas
`PUT /bank/credentials`	Membre	`eb_application_id`, `private_key_pem`	204 ; 422 si la clé n'est pas une clé RSA valide ; la clé n'est jamais renvoyée
`POST /bank/authorizations`	Membre	—	200 : `url` de redirection vers la banque ; 409 si une autorisation est déjà en cours ; 502
`GET /bank/callback`	Membre (le navigateur revient de la banque)	`code`, `state`	302 vers `/reglages/banque` ; 400 si `state` est inconnu, expiré ou déjà utilisé

Méthode et route	Rôle	Entrée	Sortie et codes
`PUT /bank/account`	Membre	`account_id`	204 ; désigne le compte suivi parmi ceux renvoyés par la banque
`POST /bank/sync`	Membre	—	202 ; 429 au-delà de 1 par heure ou 4 sur 24 heures (EF-10)
`DELETE /bank/connection`	Membre	—	204 (EF-08)
`GET /admin/users`	Administrateur	—	200 : `id`, `email`, `display_name`, `role`, `is_active`, état de la connexion
`POST /admin/users`	Administrateur	`email`, `display_name`	201 ; 409 si l'email existe ; le rôle est toujours `member`
`PATCH /admin/users/{id}`	Administrateur	`is_active`	200 ; 409 si l'administrateur se désactive lui-même

Exemple de réponse de `GET /dashboard`

```
{
  "total_saved_cents": 18420,
  "month_saved_cents": 1235,
  "transferred_cents": 15000,
  "to_transfer_cents": 3420,
  "month": "2026-11",
  "goal": { "name": "Apport", "target_cents": 200000, "progress_percent": 9,
"target_date": "2027-09-01" },
  "bank": { "status": "active", "expires_in_days": 112, "last_sync_ok_at":
"2026-11-07T05:30:12Z" }
}
```

`goal` vaut `null` si aucun objectif n'est défini ; `bank` vaut `null` si aucune connexion n'existe.

8. Écrans et parcours

L'application a 8 écrans. La navigation principale est une barre d'onglets en bas avec 4 entrées : Accueil, Transactions, Virements, Réglages. L'objectif s'ouvre depuis l'accueil ; l'administration s'ouvre depuis les réglages et n'est visible que pour l'administrateur.

L'application s'appelle Arrondi (décision D-19). Les maquettes des 8 écrans, avec l'état de l'écran Banque pour chaque banque, sont dans Arrondi : maquettes des écrans. Les couleurs, la typographie et le logo sont définis dans Arrondi : charte graphique et repris ci-dessous.

Id	Écran et route	Contenu	États particuliers
E-01	Accueil `/`	Total épargné, épargne du mois, montant à virer avec le bouton « J'ai viré », progression de l'objectif, date de la dernière synchronisation réussie	Sans connexion bancaire : message et bouton « Connecter ma banque ». Consentement expirant : bandeau orange. Consentement expiré : bandeau rouge et bouton « Reconnecter ». Hors ligne : bandeau « Hors ligne »
E-02	Transactions `/transactions`	Liste par jour avec un sélecteur de mois ; par ligne : libellé, montant, arrondi final (par exemple `+0,50 €`) ou raison d'exclusion en grisé	Aucune transaction : « Aucune transaction ce mois-ci ». Chargement par pages de 50
E-03	Virements `/virements`	Historique mensuel sur 12 mois (épargne et virement), liste des virements, bouton « Enregistrer un virement » ouvrant un formulaire (montant prérempli avec le montant à virer, date du jour)	Aucun virement : « Aucun virement enregistré »
E-04	Objectif `/objectif`	Formulaire nom, montant cible, date cible facultative ; progression ; bouton « Supprimer l'objectif »	Aucun objectif : formulaire vide

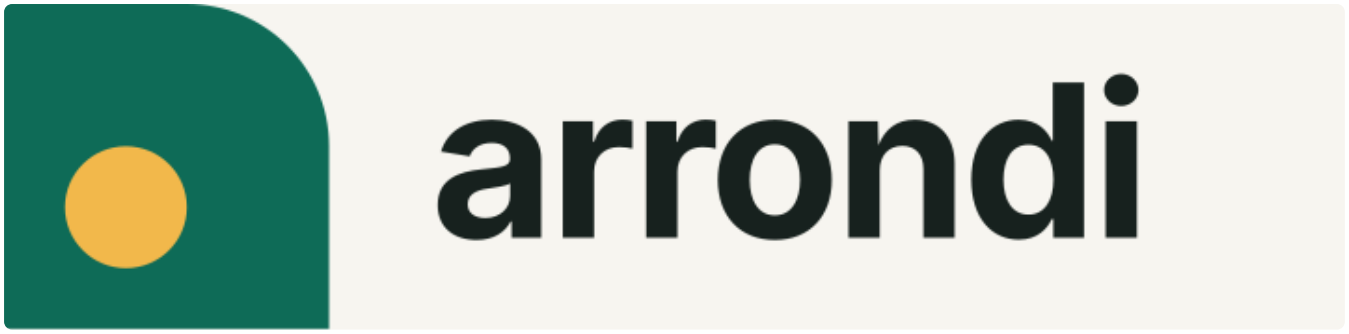
Id	Écran et route	Contenu	États particuliers
E-05	Réglages `/reglages`	Palier (1, 2, 5, 10 € ou personnalisé), multiplicateur, plafond par transaction, plafond mensuel ; liens vers la banque, le compte et l'administration	Valeur refusée : message sous le champ, rien n'est enregistré
E-06	Banque `/reglages/banque`	Assistant en 4 étapes (section 9.1 puis 9.2), état de la connexion et date de fin du consentement, choix du compte suivi, boutons « Synchroniser maintenant » et « Déconnecter »	Un message d'erreur clair par cas de la section 9.4
E-07	Compte `/reglages/compte`	Bouton « Exporter mes données », bouton « Supprimer mon compte »	Suppression : l'utilisateur doit taper le mot « SUPPRIMER »
E-08	Administration `/admin`	Liste des utilisateurs (nom, email, rôle, actif, état de la connexion), formulaire d'ajout, bouton activer ou désactiver	Visible uniquement pour l'administrateur ; sinon 404

Contraintes d'interface

- Zones tactiles d'au moins 44 par 44 points.
- Contraste de texte conforme au niveau AA des WCAG (4,5 pour 1 au minimum).
- Mode clair et mode sombre suivent automatiquement le réglage du téléphone.
- Les zones sûres de l'iPhone (encoche, barre de navigation) sont respectées.
- Les tailles de texte sont exprimées en unités relatives pour suivre le réglage de taille de l'appareil.
- Aucune notification push en version 1.

Charte graphique

La charte est la référence pour toute valeur visuelle : aucune couleur, taille ou rayon ne figure dans le code en dehors de ces jetons. Signature : « Ce que tu arrondis, tu le gardes. » Ton : tutoiement, phrases courtes et directes.



Jeton	Clair	Sombre	Usage
Surface	`#F7F5F0`	`#101614`	Fond de l'écran
Surface relevée	`#FFFFFF`	`#1A2320`	Cartes et formulaires
Texte	`#17211E`	`#EDF3F0`	Texte principal
Texte atténué	`#5A6762`	`#9DAEA7`	Texte secondaire
Primaire	`#0E6B57`	`#5CD1AE`	Actions, logo
Pièce	`#F2B84B`	`#F2B84B`	Accent du logo et des arrondis
Avertissement	`#9A4A06`	`#F0A14B`	Consentement qui expire
Danger	`#B3261E`	`#FF8A80`	Consentement expiré, erreurs
Ligne	`#76827C`	`#71847B`	Bordures et séparateurs

Tous les couples texte et fond atteignent au moins 4,5 pour 1, et les lignes au moins 3 pour 1, dans les deux modes.

- Typographie : la police système de l'appareil. Les montants utilisent des chiffres de largeur fixe. Montant principal : 44 sur 48, gras. Titre : 24 sur 30. Intertitre : 17 sur 24. Texte : 15 sur 22. Étiquette : 13 sur 18. Légende : 12 sur 16.
- Espacements : 4, 8, 12, 16, 24 et 32. Rayons : 8, 14 et 24, plus 999 pour les pastilles.
- Logo : un carré dont seul l'angle supérieur droit est arrondi, avec une pièce jaune. Sur fond clair le carré est vert ; sur fond sombre il prend le vert clair. Version monochrome pour les cas sans couleur.
- Icônes de l'application installée : 192 et 512 pixels, déclarées dans le manifeste de la PWA. Couleur de thème du manifeste : la surface claire.

9. Intégration bancaire

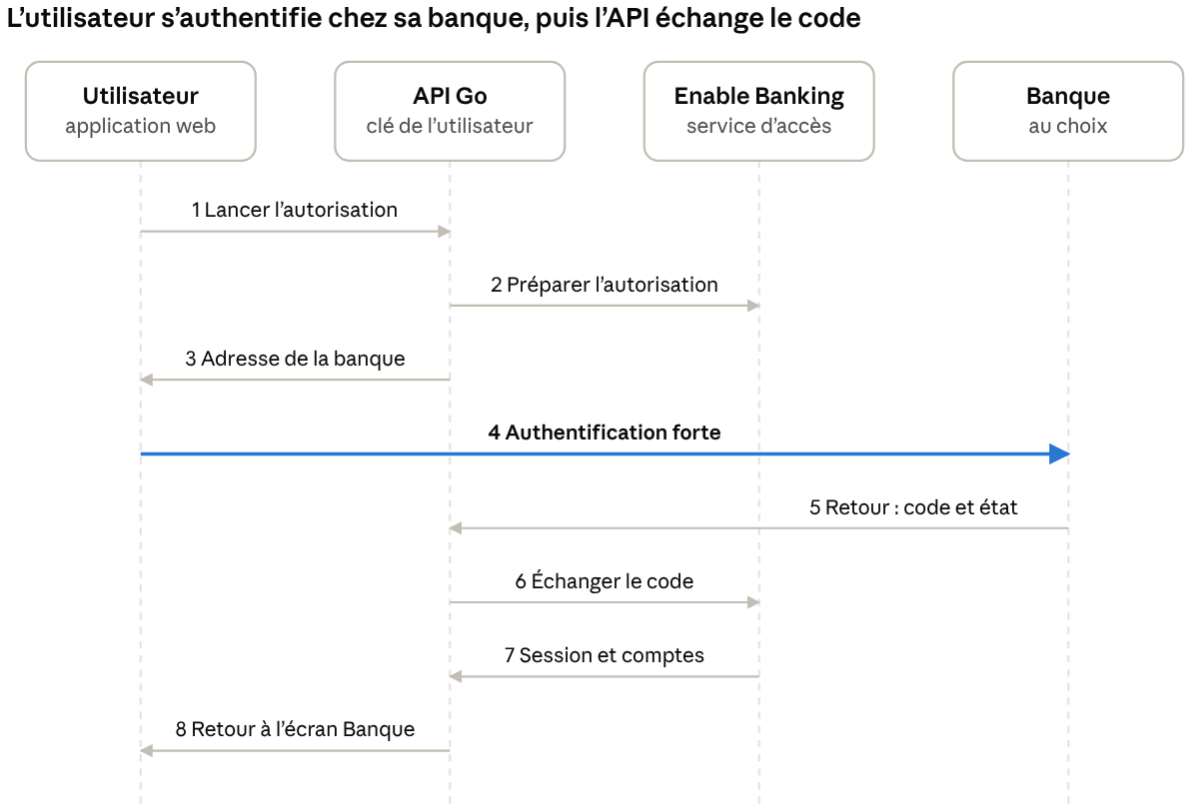
L'API lit les transactions des banques de la section 9.6 par Enable Banking, en lecture seule. Elle n'initie jamais de paiement. Détails du fournisseur : référence de l'API et démarrage rapide. Les noms exacts des champs renvoyés par chaque banque sont consignés dans une fiche de mapping par banque (section 9.6).

9.1 Préparation par chaque membre (une seule fois)

1. Créer un compte sur enablebanking.com et ouvrir le panneau de contrôle.
2. Enregistrer une application de production ; télécharger la clé privée RSA (fichier `.pem`) et noter l'identifiant de l'application.
3. Déclarer l'URL de redirection `https://<domaine>/api/v1/bank/callback` (domaine : D-14).
4. Activer l'application en liant son propre compte bancaire, ce qui la place en mode restreint (exemple d'intégration personnelle).
5. Dans l'application, écran E-06, saisir l'identifiant et le contenu du fichier `.pem`.

9.2 Flux d'autorisation

1. `POST /bank/authorizations` : l'API signe un JWT en RS256 avec la clé de l'utilisateur. L'en-tête porte `kid` égal à l'identifiant d'application ; le corps porte `iss` égal à `enablebanking.com`, `aud` égal à `api.enablebanking.com`, `iat` et `exp` égal à `iat` plus 3 600 secondes.
2. L'API cherche la banque choisie (section 9.6) dans `GET /aspsps` (pays `FR`) pour lire la durée maximale de consentement, puis appelle `POST /auth` avec la banque, l'URL de redirection, un état aléatoire de 32 octets et une fin de validité égale à cette durée maximale (exemple de flux).
3. L'API renvoie l'adresse obtenue ; l'application y redirige l'utilisateur, qui s'authentifie chez sa banque.
4. La banque renvoie l'utilisateur sur `GET /bank/callback?code=…&state=…`. L'API vérifie que l'état existe, appartient à cet utilisateur, n'a pas expiré (10 minutes) et n'a pas déjà servi, puis l'efface.
5. L'API appelle `POST /sessions` avec le code, enregistre la session, la fin du consentement et les comptes. Si `activated_at` est nul, elle le fixe à l'instant présent. L'état passe à `active`.
6. L'utilisateur choisit le compte suivi (`PUT /bank/account`).



flux d'autorisation bancaire · 8 étapes, 4 acteurs

Les étapes 1 à 8 du schéma correspondent une à une aux étapes de la liste ci-dessus ; l'étape 4 est la seule où l'utilisateur s'identifie auprès de la banque.

9.3 Synchronisation

- Déclenchement : chaque jour à 06:30 (Europe/Paris) et sur demande (EF-09, EF-10). Les synchronisations planifiées comptent dans le plafond de 4 par 24 heures, qui reste en dessous de la limite de 4 lectures quotidiennes des banques (Anchor Terminal).
- Lecture : `GET /accounts/{id}/transactions` pour le compte suivi, avec `transaction_status` égal à `BOOK`, `date_from` égal au plus tard des deux dates suivantes (jour d'activation, date de la dernière transaction connue moins 7 jours), `date_to` égal à aujourd'hui, et lecture de toutes les pages par `continuation_key`.
- Recouvrement : les 7 jours relus rattrapent les écritures tardives ; l'unicité (`account_id`, `external_id`) empêche tout doublon.
- Écriture : pour chaque transaction nouvelle, l'API évalue l'éligibilité puis, si elle est éligible, calcule et insère l'arrondi, dans la même transaction SQL que la transaction bancaire. En cas d'échec, tout le lot est annulé.

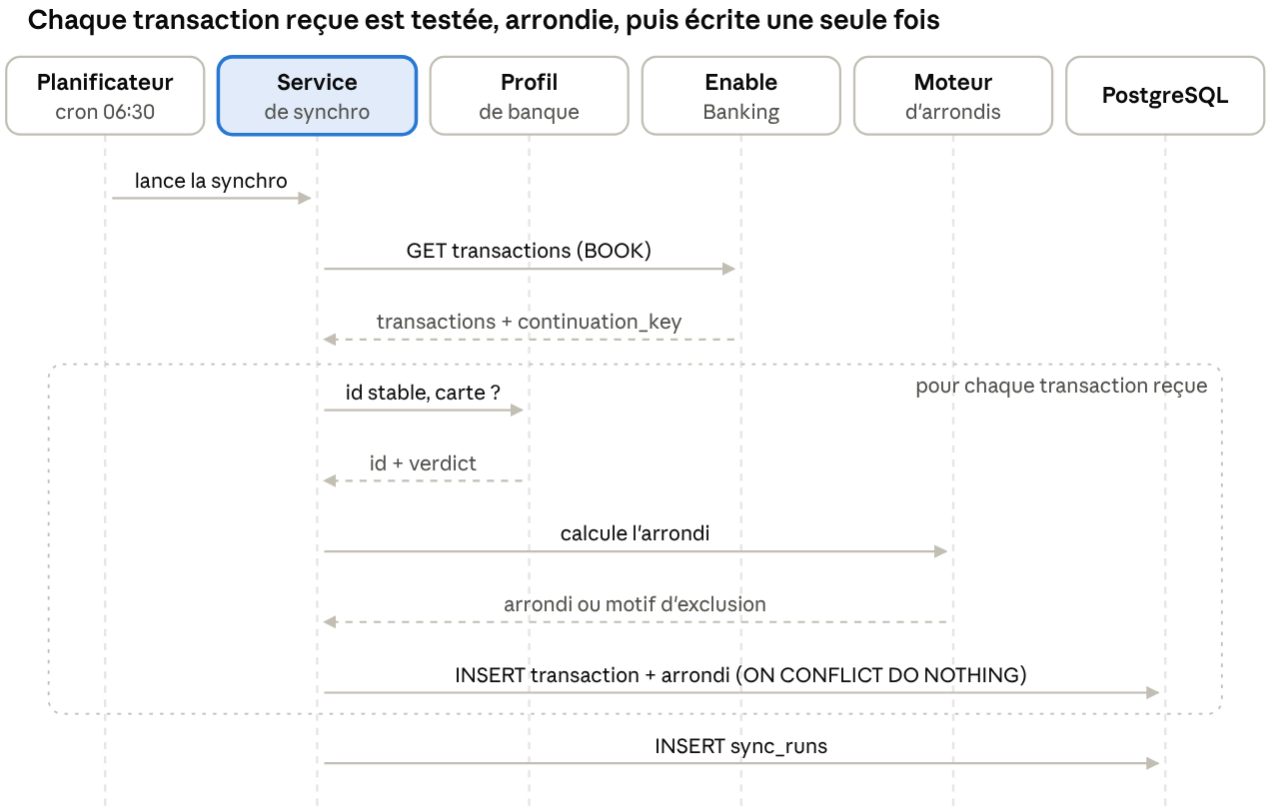


diagramme de séquence S2 · synchronisation quotidienne, 6 participants

Diagramme S2. Pour chaque transaction reçue, le profil de banque fournit l'identifiant stable et dit s'il s'agit d'un paiement par carte, puis le moteur calcule l'arrondi. L'insertion ignore les doublons grâce à `ON CONFLICT DO NOTHING`.

9.4 Gestion des erreurs

Situation	Comportement
Refus 401 ou 403 de la banque (consentement expiré ou révoqué)	Statut `expired`, synchronisations arrêtées, bandeau rouge de ré-autorisation
Réponse 429 (trop de lectures)	Pas de nouvelle tentative avant le prochain créneau ; l'exécution est notée en erreur
Erreur serveur ou délai dépassé (30 secondes)	3 tentatives espacées de 1, 5 puis 15 minutes, puis erreur
JWT refusé (identifiant ou clé invalide)	Statut `error`, message « Identifiant ou clé refusés par Enable Banking »
Réponse inattendue ou illisible	Statut `error`, aucune donnée modifiée (EF-13)

9.5 Fin de consentement

Quand la fin de validité est dans 14 jours ou moins, l'état devient « expire bientôt » et l'accueil affiche un bandeau orange. Une fois la date passée, l'état est « expirée » : les synchronisations s'arrêtent, les données déjà enregistrées restent consultables. La ré-autorisation reprend le flux 9.2 sans modifier `activated_at`.

9.6 Banques prises en charge et profils de banque

La version 1 prend en charge les quatre banques ci-dessous. Chacune est portée par un profil de banque : un module Go qui isole tout ce qui varie d'une banque à l'autre (nom attendu dans `GET /aspsps`, détection d'un paiement par carte, identifiant stable d'une transaction, durée de consentement). Le reste de l'application ne connaît aucune banque par son nom.

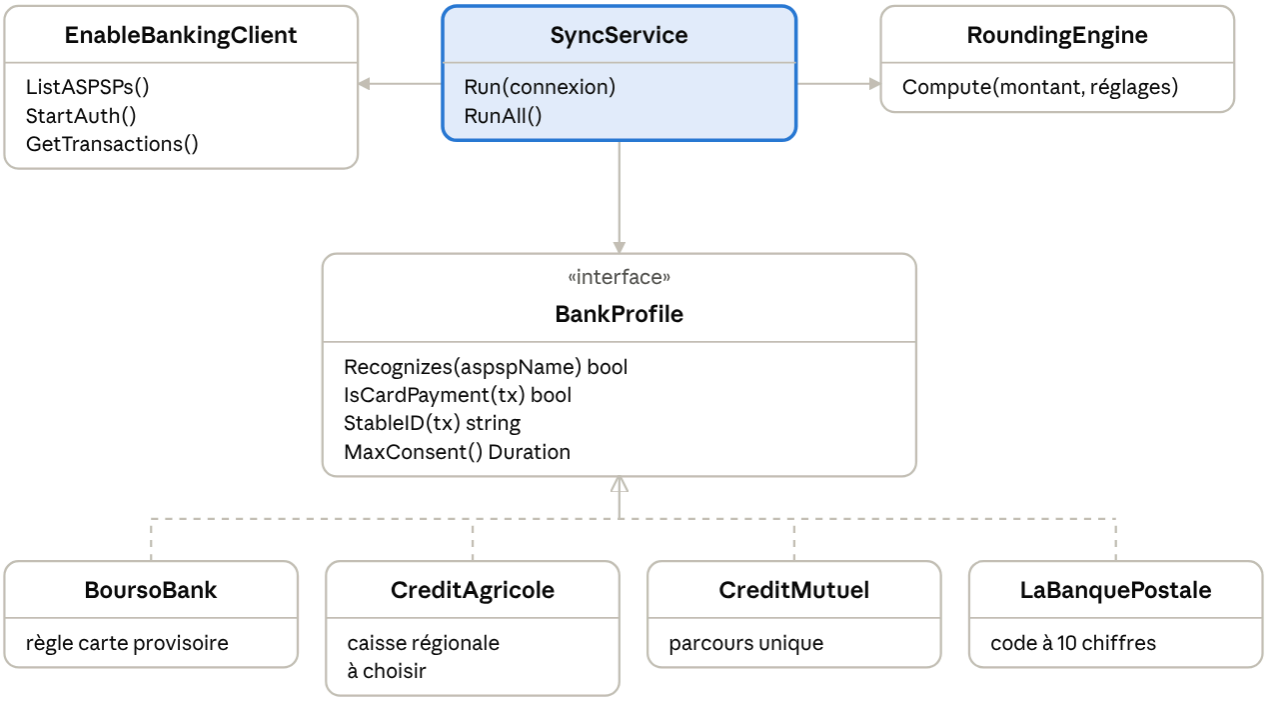
Banque	Entrée dans GET /aspsps	Particularités d'authentification	Validation avant activation
BoursoBank	Boursorama (FR)	À relever sur le compte de l'auteur	Fiche de mapping établie au jalon J1
Crédit Agricole	Une entrée par caisse régionale ; l'utilisateur choisit sa caisse locale	Authentification dans l'application Ma Banque, commune à toutes les caisses	Fiche de mapping établie avec un compte réel
Crédit Mutuel	Crédit Mutuel (FR)	Un seul parcours pour toutes les caisses régionales	Fiche de mapping établie avec un compte réel
La Banque Postale	La Banque Postale (FR)	Redirection dans le navigateur, saisie du code d'identification à 10 chiffres, pas de bascule automatique vers l'application	Fiche de mapping établie avec un compte réel

Source des particularités : Open Banking Specifics in France, Enable Banking.

- Un profil expose quatre opérations : reconnaître le nom de la banque dans `GET /aspsps`, dire si une transaction est un paiement par carte, fournir l'identifiant stable d'une transaction, donner la durée maximale du consentement.
- Un profil n'est activable que si sa fiche de mapping est complète : identifiant stable, sens du montant, statut, date de comptabilisation, règle de détection d'une carte et d'un retrait d'espèces, durée réelle du consentement, nombre de lectures acceptées par jour. Sinon la banque est affichée « Bientôt disponible » (EF-37).

- Le corps de `POST /bank/authorizations` porte `aspsp_name` et `aspsp_country`. Pour Crédit Agricole, `aspsp_name` est la caisse régionale choisie par l'utilisateur.
- Ajouter une banque revient à ajouter un profil, sa fiche et ses tests, sans changer le schéma ni les écrans. Les autres banques couvertes en France par Enable Banking (CIC, LCL, Caisse d'Épargne, Banque Populaire) sont hors périmètre de la version 1.
- La présence réelle des quatre banques dans le panneau de contrôle d'Enable Banking en 2026 reste à constater (risque R2 de l'étude de marché).

Un profil par banque : le reste de l'application ne connaît que BankProfile



Flèche pleine : appelle. Flèche creuse en pointillé : implémente.

diagramme de classes des services et profils de banque · 8 classes

Diagramme de classes C2. Le service de synchronisation ne connaît que l'interface `BankProfile` ; ajouter une banque revient à écrire une cinquième implémentation et son test unitaire.

10. Sécurité et confidentialité

L'application stocke l'historique d'achats bancaires de plusieurs personnes et, pour chacune, une clé privée qui ouvre l'accès à ses données bancaires. Les mesures ci-dessous sont obligatoires.

Sujet	Exigence
Authentification	Cloudflare Access avec code à usage unique envoyé par email, limité à la liste des emails autorisés. Durée de session : 7 jours.
Vérification par l'API	L'API valide le JWT de l'en-tête `Cf-Access-Jwt-Assertion` : signature (clés publiques du domaine d'équipe Cloudflare, mises en cache 1 heure), audience et expiration. Elle n'a aucune autre forme d'authentification.
Accès direct	L'API n'écoute que sur le réseau interne de Docker. Aucun port n'est publié sur la box ; le seul chemin est le tunnel Cloudflare.
Clé privée Enable Banking	Chiffrée en AES-256-GCM avec un nonce de 12 octets unique par chiffrement. La clé maître (32 octets) est un secret Docker en lecture seule, absent de la base et des sauvegardes, et conservée séparément par l'auteur dans son gestionnaire de mots de passe. La clé privée n'est jamais renvoyée par l'API ni écrite dans un journal.
Transport	TLS jusqu'à Cloudflare, puis tunnel chiffré jusqu'au Mac mini. Le trafic interne à Docker est en HTTP sur un réseau isolé.
Requêtes modifiant des données	Elles doivent porter `Content-Type: application/json` et l'en-tête `X-Requested-With: epargne`, sinon 400. La seule route GET qui a un effet, `/bank/callback`, est protégée par l'état aléatoire.
En-têtes de sécurité	`Content-Security-Policy` avec `default-src 'self'`, `X-Content-Type-Options: nosniff`, `Referrer-Policy: no-referrer`.
Limites	60 requêtes par minute et par utilisateur (429 au-delà) ; corps de requête de 64 Ko au maximum.
Conteneur de l'API	Exécuté sans droits administrateur, système de fichiers en lecture seule, aucune capacité Linux supplémentaire.

Sujet	Exigence
Journaux	Format JSON. Ni libellé de transaction, ni montant, ni email, ni clé, ni JWT : uniquement des identifiants d'utilisateur et des codes d'événement. Conservation de 30 jours.
Minimisation	L'IBAN complet n'est jamais stocké (4 derniers caractères). Les transactions sont conservées 24 mois (D-10), le journal d'audit 12 mois.
Droits de l'utilisateur	Export (EF-29) et suppression complète (EF-30) disponibles dans l'application. Chaque membre consent explicitement à la connexion de son compte à l'écran E-06, qui liste les données lues (transactions, libellés, quatre derniers caractères de l'IBAN).
Sauvegardes	`pg_dump` chiffré avec `age` ; seule la clé publique reste sur le Mac mini. Copie hors de la machine.
Vulnérabilités	`govulncheck` et `npm audit` exécutés avant chaque déploiement ; images reconstruites au moins une fois par mois.

11. Exigences non fonctionnelles

Volume de dimensionnement : 6 utilisateurs, 150 transactions par mois chacun, 24 mois de conservation, soit 21 600 transactions au maximum. Les valeurs ci-dessous sont des seuils à respecter ; celles marquées « à mesurer » sont vérifiées au jalon indiqué.

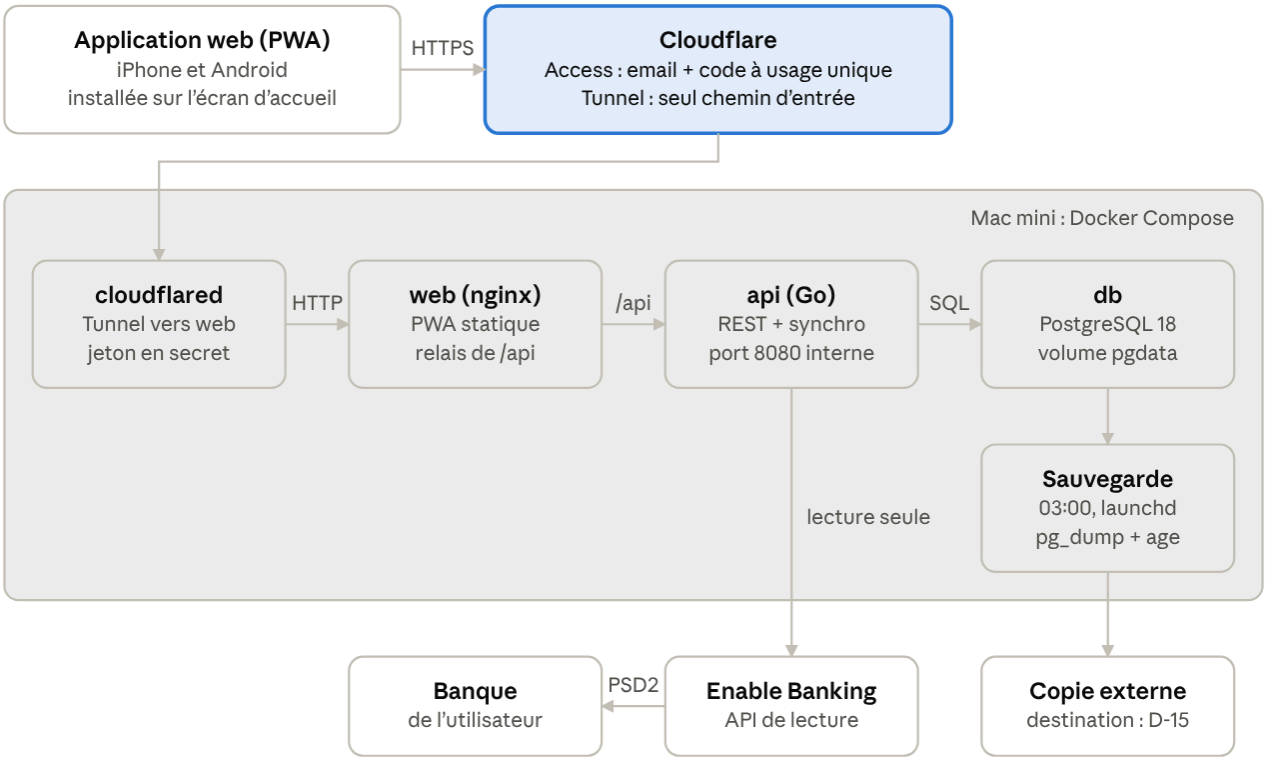
Id	Domaine	Exigence	Vérification
ENF-01	Performance de l'API	95 % des réponses de `GET /dashboard` et `GET /transactions` en moins de 300 ms avec 21 600 transactions en base	Test sur un jeu de données de ce volume, jalon J3
ENF-02	Performance de l'application	Affichage du plus grand élément en moins de 2,5 secondes en 4G sur iPhone ; script initial de moins de 200 Ko compressés	Mesure Lighthouse, jalon J4

Id	Domaine	Exigence	Vérification
ENF-03	Disponibilité	99 % par mois, soit au plus 7,2 heures d'indisponibilité sur 30 jours, sans engagement formel	Observation sur le premier mois en production
ENF-04	Sauvegarde	Une sauvegarde chiffrée par jour à 03:00 (Europe/Paris) ; 30 sauvegardes quotidiennes et 12 mensuelles conservées	Contrôle du dossier de sauvegarde, jalon J5
ENF-05	Reprise	Perte de données maximale de 24 heures ; remise en service en moins de 4 heures	Test de restauration complet avant la mise en ligne, puis chaque trimestre
ENF-06	Empreinte de l'API	Image Docker de moins de 50 Mo ; mémoire de moins de 100 Mo au repos	À mesurer au jalon J1 puis J5
ENF-07	Empreinte de la stack	Moins de 400 Mo de mémoire au repos pour les 4 conteneurs	À mesurer au jalon J5
ENF-08	Compatibilité	iOS 16.4 ou plus récent (application installée depuis Safari) ; Android avec les 2 dernières versions majeures de Chrome	Test sur au moins un appareil de chaque plateforme (iPhone, Android), jalon J6
ENF-09	Accessibilité	Contraste et taille des zones tactiles conformes à la section 8	Revue des écrans, jalon J4
ENF-10	Qualité du code	Couverture de tests d'au moins 90 % pour le package du moteur d'arrondis ; 100 % des 12 exemples de la section 4.4 sous forme de tests	Rapport de couverture, jalon J2
ENF-11	Journaux	Rotation Docker : 3 fichiers de 10 Mo au maximum par conteneur	Configuration de la stack, jalon J5
ENF-12	Fuseau horaire	Conteneurs en `TZ=Europe/Paris` ; horodatages stockés en UTC	Test sur le passage à l'heure d'hiver et d'été des mois et de la tâche de 06:30

12. Architecture et déploiement

Quatre conteneurs tournent sur le Mac mini et le tunnel Cloudflare est l'unique chemin d'entrée depuis Internet.

Le tunnel Cloudflare est le seul chemin vers les quatre conteneurs



architecture de déploiement · 4 conteneurs, 3 services externes

Le téléphone ne parle qu'à Cloudflare ; seul `cloudflared` relie Internet au réseau Docker. L'API est la seule à appeler Enable Banking.

Service	Image	Rôle	Réseau	Données
`db`	`postgres:18`	Base de données	Réseau interne `backend`, aucun port publié	Volume `pgdata`
`api`	Construite localement : binaire Go sur une image minimale	API REST, synchronisation, purge	Réseaux `backend` et `edge`, écoute sur le port 8080 en interne	Aucune écriture disque ; clé maître en secret

Service	Image	Rôle	Réseau	Données
`web`	`nginx` officielle	Sert les fichiers statiques de la PWA et relaie `/api/` vers `api:8080` en conservant l'en-tête `Cf-Access-Jwt-Assertion`	Réseau `edge`, port 80 interne	Aucune
`cloudflared`	`cloudflare/cloudflared` officielle	Tunnel vers `web:80`	Réseau `edge`	Jeton du tunnel en secret

La route `/healthz` n'est pas relayée par `web` : elle n'est joignable que depuis le réseau Docker. Les ports 80 et 443 de la box restent réservés aux portfolios publics.

Variables de configuration de l'API

Variable	Valeur	Remarque
`DATABASE_URL`	Adresse de la base	Secret
`CF_ACCESS_TEAM_DOMAIN`	Domaine d'équipe Cloudflare	Sert à trouver les clés publiques
`CF_ACCESS_AUD`	Identifiant d'audience de l'application Access	Contrôlé sur chaque JWT
`MASTER_KEY_FILE`	`/run/secrets/master_key`	Clé de 32 octets
`PUBLIC_BASE_URL`	`https://<domaine>`	Sert à construire l'URL de redirection (D-14)
`SYNC_CRON`	`CRON_TZ=Europe/Paris 30 6 * * *`	Synchronisation quotidienne
`TZ`	`Europe/Paris`	Fuseau du conteneur
`LOG_LEVEL`	`info`	

Déploiement

- La stack Compose est gérée dans Portainer à partir du dépôt Git.
- Les migrations `goose` s'exécutent au démarrage de l'API, avant qu'elle n'accepte des requêtes.

- Une mise à jour reconstruit les images puis relance la stack. Les 3 dernières images de l'API restent disponibles, ce qui permet un retour arrière en redéployant la précédente.
- Cloudflare : un tunnel nommé et une application Access sur le domaine, avec une règle « Autoriser » limitée à la liste des emails.
- Sauvegarde : une tâche `launchd` du Mac mini exécute chaque jour à 03:00 `pg_dump` dans le conteneur `db`, chiffre le résultat avec `age`, l'écrit dans le dossier de sauvegarde puis le copie hors de la machine (destination : D-15).

13. Plan de tests et critères d'acceptation

La version 1 est terminée quand toutes les exigences « Doit » des sections 5 et 11 sont vérifiées et que le jalon J6 est réussi. Chaque jalon a un critère de réussite binaire : il est atteint ou non.

Jalon	Contenu	Critère de réussite	Dépend de
J1 Preuve de concept	Programme Go qui signe le JWT, liste les banques, démarre une autorisation, crée la session et lit les transactions du compte BoursoBank de l'auteur	(1) BoursoBank figure dans `GET /aspsps` pour `FR` ; (2) le consentement est obtenu avec la durée maximale annoncée ; (3) au moins 30 jours de transactions sont lus ; (4) la fiche de mapping est rédigée : identifiant stable, sens, statut, date de comptabilisation, détection d'un paiement par carte et d'un retrait, durée réelle du consentement, nombre de lectures acceptées par jour ; (5) mémoire et taille d'image mesurées	Aucun
J2 Moteur d'arrondis	Package Go pur, sans accès à la base	Les 12 exemples de la section 4.4 passent ; couverture d'au moins 90 % ; propriétés de la liste ci-dessous vérifiées	J1

Jalon	Contenu	Critère de réussite	Dépend de
J3 API et base	Migrations, routes, authentification, chiffrement, synchronisation	EF-01 à EF-18, EF-35 et ENF-01 vérifiés	J2
J4 Application web	Les 8 écrans, installation, hors ligne limité	EF-19 à EF-34, ENF-02 et ENF-09 vérifiés	J3 (contrat OpenAPI)
J5 Déploiement	Stack Compose, tunnel, Access, sauvegardes	Restauration complète réussie sur une base vide ; ENF-04 à ENF-07 et ENF-11 vérifiés	J3
J6 Recette utilisateurs	Installation chez tous les utilisateurs actifs	Toutes les installations réussissent ; chaque membre voit sur un achat réel un arrondi égal à celui qu'il calcule à la main ; ENF-08 vérifiée	J4 et J5

J4 et J5 peuvent être menés en parallèle. Si le critère (1) ou (2) de J1 échoue, le projet s'arrête ou bascule vers la solution de repli décrite dans l'étude de marché : aucun autre jalon ne démarre.

Propriétés vérifiées sur le moteur d'arrondis (J2)

- L'arrondi final n'est jamais négatif.
- L'arrondi final ne dépasse jamais le plafond par transaction.
- L'arrondi de base est toujours strictement inférieur au palier.
- Si le montant est un multiple exact du palier, l'arrondi de base vaut 0.
- Montant plus arrondi de base est toujours un multiple du palier.
- Avec un plafond mensuel, la somme des arrondis finaux d'un mois ne le dépasse jamais.

Tests par niveau

Niveau	Outil	Ce qui est testé
Unitaire Go	Package `testing`, tests par tableau	Moteur d'arrondis ; une ligne par code d'exclusion de la section 4.2 ; chiffrement (aller-retour, nonce différent à chaque appel, donnée modifiée détectée) ; découpage des mois autour des changements d'heure du 29 mars et du 25 octobre 2026
Intégration Go	PostgreSQL réel en conteneur, faux serveur Enable Banking	Migrations ; contraintes `CHECK` ; isolation entre utilisateurs (EF-03) ; synchronisation rejouée deux fois sans doublon (EF-11) ; flux d'autorisation et chaque cas de la section 9.4 ; purge de 24 mois
API	Tests de requêtes	JWT absent, expiré, mal signé ou d'une autre audience (EF-01) ; email inconnu (EF-02) ; limites de débit
Application	Vitest pour les composants, Playwright de bout en bout sur deux formats d'écran (iPhone et Android)	Formatage des montants en français ; parcours E-01 à E-08 ; manifeste installable ; mode hors ligne limité
Recette	Liste de contrôle sur appareils réels	Section J6

14. Hors périmètre et évolutions

Les points suivants ne font pas partie de la version 1 et ne doivent pas être développés.

- Tout virement automatique vers l'épargne et toute initiation de paiement.
- Une application native iOS ou Android et toute publication sur un magasin d'applications.
- Plusieurs comptes suivis par utilisateur, et toute banque absente de la liste de la section 9.6.
- Toute devise autre que l'euro.
- La catégorisation des dépenses, les budgets, les graphiques et les statistiques avancées.
- Les notifications push et les emails.
- Un objectif ou une épargne partagés entre plusieurs membres.

- La saisie manuelle de transactions et l'import de relevé, sauf si les risques R1 ou R2 de l'étude de marché se réalisent.
- La supervision automatique et les alertes : l'auteur consulte Portainer.
- Plusieurs langues.
- Le recalcul rétroactif des arrondis.

Évolutions envisagées après la version 1 : import d'un relevé exporté, plusieurs objectifs, notifications, application native sur la même API.

15. Décisions à valider

Chaque décision ci-dessous a une valeur retenue qui s'applique dès maintenant. Une décision non contestée avant le début du jalon qui l'utilise est considérée comme validée. D-14 et D-15 n'ont pas de valeur par défaut : elles attendent une information de l'auteur.

Id	Décision retenue	Alternative possible	Utilisée dans
D-01	Une application Enable Banking par utilisateur, avec clé privée stockée chiffrée	Une application commune pour tous : contraire aux conditions d'utilisation, avec risque de désactivation	9, 10
D-02	Quand le montant est déjà un multiple exact du palier, l'arrondi est nul	Arrondir au palier suivant, comme Trade Republic où 2 € devient 3 € (source)	4.3
D-03	Multiplicateurs permis : 1, 2, 3, 5, 10	D'autres valeurs entières	4.1
D-04	Palier personnalisé de 0,50 € à 50,00 € ; plafond par transaction de 20,00 € par défaut, réglable de 1 à 100 €	Aucun plafond par transaction	4.1
D-05	Plafond mensuel désactivé par défaut	Activé par défaut à 100,00 €	4.1

Id	Décision retenue	Alternative possible	Utilisée dans
D-06	Détection d'un paiement par carte : une règle par banque, portée par un profil de banque (section 9.6). Pour BoursoBank, règle provisoire « le libellé commence par `CARTE` », hypothèse non vérifiée. Chaque règle est confirmée ou remplacée par la fiche de mapping de sa banque ; une banque sans règle validée n'est pas activable	S'appuyer sur le code de transaction bancaire	4.2, 13
D-07	Un remboursement n'a aucun effet sur les arrondis enregistrés	Annuler l'arrondi de la transaction d'origine	4.5
D-08	Aucun recalcul rétroactif après un changement de réglage	Recalculer tout l'historique	4.5
D-09	Le comptage commence à la date d'activation, sans historique antérieur	Compter depuis le premier jour du mois en cours	4.2
D-10	Transactions conservées 24 mois	12 ou 36 mois	10, EF-31
D-11	Versions minimales : iOS 16.4, Android avec les 2 dernières versions majeures de Chrome	À ajuster selon les appareils réels des utilisateurs	2, ENF-08
D-12	Isolement strict entre membres, y compris vis-à-vis de l'administrateur	L'administrateur voit des totaux par membre	2, 7
D-13	Synchronisation quotidienne à 06:30 (Europe/Paris)	Un autre horaire	9.3
D-14	**À fournir par l'auteur** : le nom de domaine (sous-domaine dédié) par lequel l'application est atteinte	Sans objet	9.1, 12
D-15	**À fournir par l'auteur** : la destination de la copie de sauvegarde hors du Mac mini	Un disque externe ou un stockage en ligne personnel	12

Id	Décision retenue	Alternative possible	Utilisée dans
D-16	Session Cloudflare Access de 7 jours	24 heures ou 30 jours	10
D-17	Une banque se branche par un profil de banque (module Go) ; quatre profils en version 1 : BoursoBank, Crédit Agricole, Crédit Mutuel, La Banque Postale	Une seule logique de détection pour toutes les banques : impossible, les libellés et identifiants diffèrent	4.2, 9.6
D-18	Une banque n'est ouverte aux utilisateurs qu'après sa fiche de mapping, établie sur un compte réel ; l'auteur valide BoursoBank au jalon J1, les autres banques sont validées avec le compte d'un utilisateur volontaire	Ouvrir les quatre banques dès le départ sans validation	9.6, 13
D-19	L'application s'appelle Arrondi ; son identité (nom, logo, couleurs, typographie) est définie par la charte de la section 8, et les écrans par les maquettes liées dans cette même section	Rester sans nom ni identité et laisser l'interface au style par défaut	8